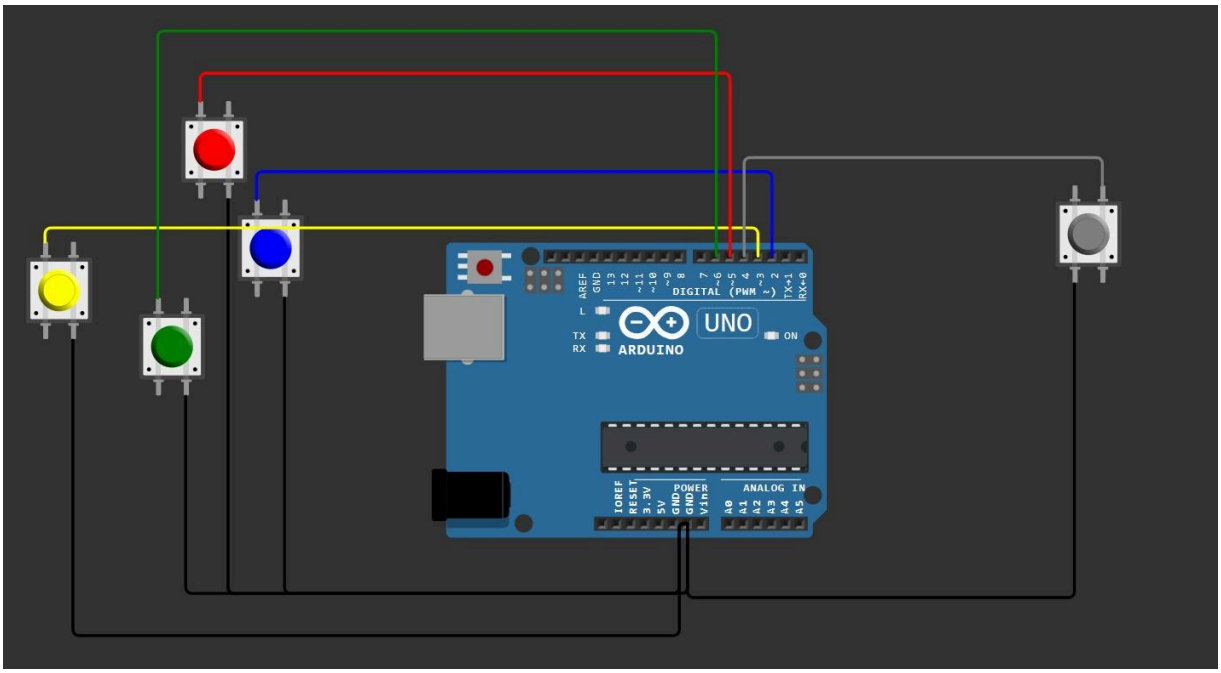
# DIY PC Gaming Controller with Arduino & Python

**Project Overview** This guide outlines how to build a custom gaming controller using an Arduino Uno. It maps physical push buttons to standard PC gaming inputs (W, A, S, D, and Space) by bridging microcontroller serial data with DirectX-level keystroke emulation in Python.

## 1. Hardware & Wiring Schematic

The controller uses the Arduino's internal INPUT_PULLUP resistors, meaning no external resistors are required.
*Schematics:*




**Breadboard Connections:** *(Refer to the schematics while doing breadboard connections)*

1. Connect a single jumper wire from the **GND** pin on the Arduino to the negative (black/blue) power rail on your breadboard.
2. Place your 5 push buttons onto the breadboard.
3. Connect **one leg** of each button to the shared Ground rail.
4. Connect the **diagonally opposite leg** of each button to the Arduino digital pins as follows:
    - **Pin D2:** Button D (Right)
    - **Pin D3:** Button A (Left)
    - **Pin D4:** Button Space (Action)
    - **Pin D5:** Button W (Forward)
    - **Pin D6:** Button S (Backward)

# 2. The Arduino Firmware (C++)

This code monitors the state of the buttons. When a button is pressed or released, it sends a short string over the Serial connection (e.g., 5P for "Pin 5 Pressed", or 5R for "Pin 5 Released").
**Instructions:** Open the Arduino IDE, paste the code below, select your board (Arduino Uno) and port, and click **Upload**.
Code:

```
const int pins[] = {2, 3, 4, 5, 6};
int lastStates[5];

void setup() {
  Serial.begin(9600);

  for (int i = 0; i < 5; i++) {
    pinMode(pins[i], INPUT_PULLUP);
    lastStates[i] = HIGH;
  }
}

void loop() {
  for (int i = 0; i < 5; i++) {
    int currentState = digitalRead(pins[i]);

    if (currentState != lastStates[i]) {
      Serial.print(pins[i]);

      if (currentState == LOW) {
        Serial.println("P");
      } else {
        Serial.println("R");
      }

      lastStates[i] = currentState;
    }
  }

  delay(10);
}
```

# 3. The Python Software Bridge

This script acts as the translator. It listens to the COM port for the Arduino's messages and uses the pydirectinput library to trick your PC games into thinking a real keyboard is being pressed.
**Prerequisites:** Ensure Python 3 is installed. Open your terminal/command prompt and run: pip

install pyserial pydirectinput
**The Code:** Create a file named controller.py and paste the following code. *(Note: Update 'COM5' to match your Arduino's active port).*

*Code:*

```
import serial
import pydirectinput

# Configuration: Map Arduino pins to Keyboard Keys
# Pin 2=D, 3=A, 4=Space, 5=W, 6=S
key_map = {
    "2": "d", "3": "a", "4": "space", "5": "w", "6": "s"
}

# Change 'COM5' to your Arduino's port!
ser = serial.Serial('COM5', 9600, timeout=1)

print("Controller Active! Press Ctrl+C to stop.")

while True:
    if ser.in_waiting > 0:
        line = ser.readline().decode('utf-8').strip()
        print(f"Arduino said: {line}")

        if len(line) >= 2:
            pin = line[:-1]   # The number (e.g., "2")
            action = line[-1] # The action ("P" for press, "R" for release)

            if pin in key_map:
                key = key_map[pin]
                if action == "P":
                    pydirectinput.keyDown(key)
                else:
                    pydirectinput.keyUp(key)
```

# 4. Running the Setup

To bring everything together and start gaming, follow these exact steps:

1. **Plug in your Arduino** and ensure the firmware is successfully uploaded.
2. **Close the Arduino IDE completely** (or at the very least, close the Serial Monitor). If the Arduino IDE is listening to the port, Python will throw an "Access is denied" error.
3. Open a fresh terminal (Command Prompt, PowerShell, or VS Code terminal).
4. Navigate to the folder where you saved controller.py.
5. Run the script by typing: python controller.py
6. You should see "Controller Active!". Open your game, start pressing your breadboard

buttons, and enjoy your custom controller.